LAPORAN PRAKTIKUM
ALGORITMA DAN STRUKTUR DATA

Oleh :

Nur Ihwan Nauval

NIM : 2025573010089

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

PROGRAM STUDI TEKNIK INFORMATIKA

POLITEKNIK NEGRI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum Algoritma dan Struktur Data ini dengan baik.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan, serta untuk memenuhi tugas yang diberikan oleh Bapak *Muhammad Reza Zulman, S>ST.,M.Sc*. Dalam laporan ini dibahas mengenai konsep kompleksitas algoritma, seperti time complexity dan space complexity, penggunaan Big-O Notation, serta penerapan refactoring dalam kode program.

Saya menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca.

Desa Tgk Dibalee 28 April 2026

Nur Ihwan Nauval

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 LATAR BELAKANG 1
1.2 TUJUAN 1
1.3 RUMUSAN MASALAH 1
BAB II PEMBAHASAN 2
2.1 PENJELASAN 2
2.1.1 INTRO KOMPLEKSITAS 2
2.1.2 KELAS BIG O 2
2.1.3 SPACE KOMPLEKXITY 3
2.1.4 REFACTOR 3
2.2 LANGKAH PRAKTIKUM 4
2.2.1 MEMBUAT FOLDER PRAKTIKUM-5 4
2.2.2 MEMBUAT FILE DAN ISINYA 4
2.2.3 LATIHAN DAN TUGAS 8
2.2.4 OUTPUTNYA 14
BAB III PENUTUP 17
3.1 KESIMPULAN 17

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Dalam pengembangan perangkat lunak, efisiensi algoritma menjadi hal yang sangat penting. Hal ini dikarenakan algoritma yang tidak efisien dapat menyebabkan program berjalan lambat dan membutuhkan banyak memori, terutama ketika menangani data dalam jumlah besar.

Oleh karena itu, diperlukan pemahaman mengenai kompleksitas algoritma, baik dari segi waktu (time complexity) maupun ruang (space complexity). Analisis ini biasanya menggunakan Big-O Notation untuk mengetahui tingkat efisiensi suatu algoritma.

Selain itu, dalam praktik pemrograman juga diperlukan teknik refactoring untuk memperbaiki struktur kode agar lebih rapi, mudah dipahami, dan lebih optimal tanpa mengubah fungsi utama program.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep-konsep tersebut serta mampu menerapkannya dalam pembuatan program sederhana.

1.2 TUJUAN

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar kompleksitas algoritma.
2. Mengetahui perbedaan time complexity dan space complexity.
3. Memahami penggunaan Big-O Notation dalam analisis algoritma.
4. Mampu mengelompokkan algoritma berdasarkan kelas kompleksitasnya.
5. Memahami konsep refactoring dalam pemrograman.
6. Mampu menerapkan konsep tersebut dalam pembuatan program sederhana.

1.3 RUMUSAN MASALAH

1. Apa yang dimaksud dengan kompleksitas algoritma?
2. Apa perbedaan antara time complexity dan space complexity?
3. Apa itu Big-O Notation dan bagaimana penggunaannya?
4. Apa saja kelas-kelas dalam Big-O?
5. Apa yang dimaksud dengan refactoring dalam pemrograman?
6. Bagaimana penerapan konsep-konsep tersebut dalam praktik coding?

BAB II

PEMBAHASAN

2.1 PENJELASAN

2.1.1 INTRO KOMPLEKSITAS

Kompleksitas algoritma adalah ukuran untuk mengetahui seberapa efisien suatu algoritma dalam menyelesaikan masalah. Efisiensi ini biasanya dilihat dari dua aspek utama, yaitu waktu eksekusi (time complexity) dan penggunaan memori (space complexity).

Time complexity digunakan untuk mengukur berapa lama waktu yang dibutuhkan algoritma untuk berjalan berdasarkan ukuran input (n). Sedangkan space complexity digunakan untuk mengukur berapa banyak memori yang digunakan selama proses eksekusi algoritma.

Dalam analisis kompleksitas, biasanya digunakan notasi khusus yang disebut Big-O Notation. Notasi ini digunakan untuk menggambarkan batas atas dari pertumbuhan waktu atau ruang yang dibutuhkan algoritma.

Beberapa contoh kompleksitas yang umum antara lain:

- $O(1)$ → waktu konstan, tidak dipengaruhi ukuran input
- $O(n)$ → waktu linear, bertambah seiring jumlah data
- $O(n^2)$ → waktu kuadrat, biasanya muncul pada algoritma dengan perulangan bersarang

Dengan memahami kompleksitas algoritma, kita dapat memilih algoritma yang lebih efisien dan sesuai dengan kebutuhan, terutama ketika berhadapan dengan data dalam jumlah besar.

2.1.2 KELAS BIG O

Dalam analisis algoritma menggunakan Big-O Notation, terdapat beberapa kelas kompleksitas yang digunakan untuk mengelompokkan algoritma berdasarkan tingkat pertumbuhan waktu eksekusinya terhadap ukuran input (n).

Berikut adalah kelas-kelas Big-O yang umum:

Kelas Big O	Penjelasan
O(1) - Konstan	Algoritma dengan kompleksitas ini memiliki waktu eksekusi yang tetap, tidak dipengaruhi oleh ukuran input.
O(log n) - Logaritmik	Waktu eksekusi bertambah secara lambat Ketika ukuran input meningkat. Biasanya terjadi pada algortima yang membagi masalah menjadi dua bagian.
O(n) - Linear	Waktu eksekusi bertambah sebanding dengan jumlah data.

O(n log n) – Linear Logaritmik	Kombinasi antara linear dan logaritmik, sering muncul pada algoritma yang menggunakan teknik divide and conquer.
O(n^2) - Kuadratik	Waktu eksekusi meningkat sangat cepat karena adanya perulangan bersarang.
O(2^n) – Eksponensial	Waktu eksekusi bertambah sangat drastis seiring bertambahnya input. Biasanya digunakan pada solusi brute force.
O(n!) - Faktorial	Merupakan kompleksitas paling tinggi dan sangat tidak efisien untuk input besar.

2.1.3 SPACE KOMPLEKXITY

Space complexity adalah ukuran yang digunakan untuk mengetahui seberapa banyak memori yang dibutuhkan oleh suatu algoritma selama proses eksekusi. Analisis ini penting karena selain waktu, penggunaan memori juga mempengaruhi efisiensi suatu program.

Space complexity biasanya dihitung berdasarkan jumlah memori tambahan (auxiliary space) yang digunakan terhadap ukuran input (n). Sama seperti time complexity, analisis ini juga menggunakan Big-O Notation.

Beberapa contoh space complexity antara lain:

Space Conplexity	Penjelasan
O(1) - Konstan	Algoritma hanya menggunakan sedikit memori tambahan dan tidak bergantung pada ukuran input.
O(n) – Linear	Penggunaan memori bertambah seiring dengan jumlah data.
O(n^2) - Kuadrat	Memori yang digunakan sangat besar karena bergantung pada dua dimensi input.

2.1.4 REFACTOR

Refactoring adalah proses memperbaiki struktur internal kode tanpa mengubah fungsi atau output dari program. Tujuan utama refactoring adalah untuk membuat kode menjadi lebih rapi, mudah dibaca, mudah dipahami, dan lebih efisien.

Dalam konteks algoritma dan struktur data, refactoring sering dilakukan untuk :

- Mengurangi kompleksitas kode

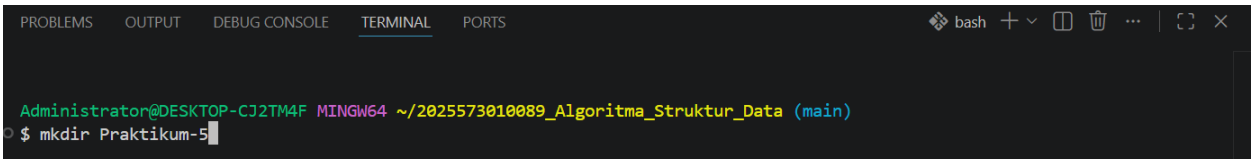
- Meningkatkan performa (time complexity dan space complexity)
- Menghilangkan redundansi (kode yang berulang)
- Mempermudah proses maintenance dan pengembangan lanjutan

Refactoring dapat berdampak langsung pada efisiensi algoritma. Misalnya, algoritma dengan kompleksitas $O(n^2)$ dapat diubah menjadi $O(n)$ dengan pendekatan yang lebih optimal. Analisis ini tetap mengacu pada Big-O Notation untuk mengukur peningkatan performa.

2.2 LANGKAH PRAKTIKUM

2.2.1 MEMBUAT FOLDER PRAKTIKUM-5

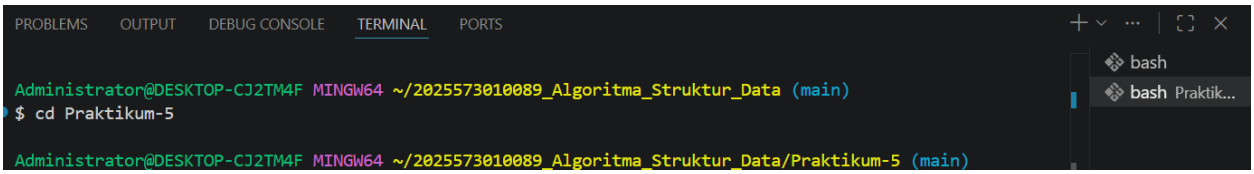
Langkah pertama, kita akan membuat folder yang bernama Praktikum-5, dan caranya buka terminal lalu kita ketik seperti gambar berikut ini.



Maka setelah kita klik enter, folder Praktikum-5 akan muncul dibagian kiri jika berhasil command tersebut.

2.2.2 MEMBUAT FILE DAN ISINYA

Selanjutnya, setelah kita membuat folder Praktikum-5, kita akan masuk ke dalam folder yang baru saja kita buat tadi, yaitu folder Praktikum-5, dan caranya kita lakukan seperti gambar dibawah.



Setelah berpindah ke dalam folder Praktikum-5, kita akan membuat beberapa file didalamnya, dan kita akan membat dan mengisi file tersebut sesuai urutannya seperti berikut.

1. Intro-Kompleksitas.js

Setelah membuat file yang pertama, kita akan mengisinya dengan beberapa code, kita akan isi seperti gambar yang ada dibawah.

```
Praktikum-5 > JS 01-intro-kompleksitas.js > cariPasangan > j
function ukurWaktu(label, fn) {
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_000; //ukuran data 100 ribu

function jumlahkanLinear(n) {
    let total = 0;
    for (let i = 1; i<= n; i++) total += i;
    return total;
}

function jumlahkanRumus(n) {
    return n * (n + 1) / 2;
}

function cariPasangan(arr) {
    const pasangan = [];
    for (let i = 0; i < arr.length; i++) {
        for (let j = i +1; j < arr.length; j++) {
            if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
        }
    }
    return pasangan;
}

const data = Array.from({length: 5000}, (_, i) => i);

console.log('=== Perbandingan Waktu Eksesekusi ===');
ukurWaktu('O(1) jumlahkanRumus', () => jumlahkanRumus(N));
ukurWaktu('O(n) jumlahkanLinear', () => jumlahkanLinear(N));
ukurWaktu('O(n^2) cariPasangan', () => cariPasangan(data));

console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
```

Setelah kita isi file intro-kompleksitas.js dengan code-code seperti diatas, kita akan lakukan hal yang sama dengan file – file lainnya.

2. Kelas-big-o.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ touch 02-kelas-big-o.js
```

Selanjutnya kita akan mengisi file kelas-big-o.js dengan code-code juga, sama seperti file yang pertama, tapi dengan code yang berbeda.

Praktikum-5 > JS 02-kelas-big-o.js > binarySearch

```javascript
function ambilPertama(arr) { return arr[0]; }
function tambahkanItem(arr, item) {arr.push(item);}
function isGenap(n) { return n % 2 === 0; }

function binarySearch(arr, target) {
    let kiri = 0, kanan = arr.length -1;
    let langkah = 0;
    while (kiri <= kanan) {
        langkah++;
        const tengah = Math.floor((kiri + kanan) / 2);
        if (arr[tengah] === target) {
            console.log(` Ditemukan di index ${tengah} setelah ${langkah} langkah`);
            return tengah;
        }
        if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}

function cariMax(arr) {
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}

function bubbleSortNaif(arr) {
    const a = [...arr];
    for (let i = 0; i < a.length; i++)
        for (let j = 0; j < a.length - i - 1; j++)
            if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j], a[j+1]];
    return a;
}

function fibRekursif(n) {
    if (n <= 1) return n;
    return fibRekursif(n-1) + fibRekursif(n-2);
}

console.log('=== O(1) - selalu cepat ===');
console.log(ambilPertama([10, 20, 30, 40, 50]));
console.log(isGenap(42));

console.log('\n=== O(log n) - Binary Search ===');
const sorted = Array.from({length: 1_000_000},(_,i) => i);
binarySearch(sorted, 731_452);

console.log('\n=== O(n) - Bubble Sort (kecil saja) ===');
console.log('Max dari 1000 elemen:', cariMax(Array.from({length: 1000},()=> Math.random()*1000|0)));

console.log('\n=== O(n²) — Bubble Sort (kecil saja) ===');
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log('\n=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===');
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
console.log('');

console.log('\nWaktu fib(35) O(2ⁿ):');
let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');
console.log('Waktu fib(35) O(n) memoization:');
const memo = {};
function fibMemo(n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
}
t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');
```

3. Space-complexity.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ touch 03-space-complexity.js
```

Isi filenya :

```
JS 03-operator.js   JS tugas-1.js Praktikum-2\...   JS 01-intro-kompleksitas.js   JS 02-kelas-big-o.js   JS 03-space-complexity.js
Praktikum-5 > JS 03-space-complexity.js > ...
1  function jumlahArray(arr) {
2      let total = 0;
3      for (const x of arr) total += x;
4      return total;
5  }
6
7  // --- O(n) Space: membuat struktur baru sebesar input ---
8  function duplikasiArray(arr) {
9      const baru = [];
10     for (const x of arr) baru.push(x * 2);
11     return baru;
12 }
13
14 // --- O(n) Space: rekursi (call stack) ---
15 function faktorialRekursif(n) {
16     if (n <= 1) return 1;
17     return n * faktorialRekursif(n - 1);
18 }
19
20 // --- O(1) Space: faktorial iteratif ---
21 function faktorialIteratif(n) {
22     let hasil = 1;
23     for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
24     return hasil;
25 }
26 const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
27 console.log('Jumlah array :', jumlahArray(arr));
28 console.log('Duplikasi array:', duplikasiArray(arr));
29 console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
30 console.log('Faktorial 10 iteratif :', faktorialIteratif(10));
31
32 // Visualisasi: hitung elemen unik (O(n) space)
33 function hitungUnik(arr) {
34     const seen = new Set();
35     for (const x of arr) seen.add(x);
36     return seen.size;
37 }
38
39 const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
40 console.log('Elemen unik:', hitungUnik(dataAcak));
41
```

4. Refactor.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ touch 04-refactor.js
```

Isi filenya :

```
Praktikum-5 > JS 04-refactor.js > ...
function adaDuplikatLambat(arr) {
    for (let i = 0; i < arr.length; i++)
        for (let j = i+1; j < arr.length; j++)
            if (arr[i] === arr[j]) return true;
    return false;

}
function adaDuplikatCepat(arr) {
    const seen = new Set();
    for (const x of arr) {
        if (seen.has(x)) return true;
    }
    return false;
}

function frekuensilambat(arr) {
    const unik = [];
    const hitung = [];
    for (const x of arr) {
        const idx = unik.indexOf(x);
        if (idx === -1) { unik.push(x); hitung.push(1); }
        else hitung[idx]++;
    }
    return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
}
function frekuensiCepat(arr) {
    const counter = {};
    for (const x of arr) counter[x] = (counter[x] || 0) + 1;
    return counter;
}

const besar = Array.from({length: 20000}, () => Math.floor(Math.random() * 1000));
let t = Date.now();
adaDuplikatLambat(besar);
console.log('Duplikat LAMBAT O(n²):', Date.now()-t, 'ms');
t = Date.now();
adaDuplikatCepat(besar);
console.log('Duplikat CEPAT O(n) :', Date.now()-t, 'ms');
const kata = ['a','b','a','c','b','a','d'];
console.log('\nFrekuensi:', frekuensiCepat(kata));
```

2.2.3 LATIHAN DAN TUGAS

1. Latihan-1.js

Sebelum membuat latihan 1, kita akan buat file untuk membuat latihan satu didalam file tersebut, dan caranya sama seperti sebelumnya.

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ touch latihan-1.js
```

Setelah kita buat file latihan-1.js, selanjutnya kita akan masuk ke filenya dengan cara klik sekali lalu didalamnya kita akan pasang code code yang sesuai apa yang disuruh di latihan.

Praktikum-5 > JS latihan-1.js > fungsiC

```javascript
function hitungKompleksitas(n, fn) {
  const start = Date.now();

  fn(n);

  const end = Date.now();

  console.log(`Waktu eksekusi untuk n=${n}: ${end - start} ms`);
}

function fungsiA(n) {
  return n * 2;
}

function fungsiB(n) {
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {}
  }
}

function fungsiC(n) {
  for (let i = 1; i < n; i *= 2) {}
}

function fungsiD(n) {
  const arr = Array.from({ length: n }, (_, i) => i);

  arr.forEach((x) => {
    arr.forEach((y) => {
      arr.forEach((z) => {});
    });
  });
}

const n = 1000;

console.log("=== Fungsi A ===");
hitungKompleksitas(n, fungsiA);

console.log("\n=== Fungsi B ===");
hitungKompleksitas(n, fungsiB);

console.log("\n=== Fungsi C ===");
hitungKompleksitas(n, fungsiC);

console.log("\n=== Fungsi D ===");
hitungKompleksitas(50, fungsiD);
```

2. Latihan 2

Berbeda dengan latihan 1, latihan 2 tidak perlu membuat file baru lagi, karna di latihannya memerintah kita untuk membuatnya di file space-complexity.js, jadi kita akan menambahkannya saja dengan latihan 2.

```javascript
//Latihan 2 :

console.log('\n=== Latihan 2 : ===\n');

function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }

  return null;
}

function cariPasanganCepat(arr, target) {
  const set = new Set();

  for (let angka of arr) {
    const selisih = target - angka;

    if (set.has(selisih)) {
      return [selisih, angka];
    }

    set.add(angka);
  }

  return null;
}

console.log("=== Uji Array Kecil ===");

const arrKecil = [2, 7, 11, 15];
const targetKecil = 9;

console.log("Lambat:", cariPasanganLambat(arrKecil, targetKecil));

console.log("Cepat:", cariPasanganCepat(arrKecil, targetKecil));

console.log("\n=== Uji Waktu Array Besar ===");

const arrBesar = Array.from({ length: 50000 }, () =>
  Math.floor(Math.random() * 100000),
);

const targetBesar = -1;

console.log("\nMengukur waktu fungsi Lambat...");

let start = Date.now();

cariPasanganLambat(arrBesar, targetBesar);

let end = Date.now();

console.log("Waktu Lambat:", end - start, "ms");

console.log("\nMengukur waktu fungsi Cepat...");

start = Date.now();

cariPasanganCepat(arrBesar, targetBesar);

end = Date.now();

console.log("Waktu Cepat:", end - start, "ms");
```

3. Tugas 1

Untuk tugas, kita akan membuat folder baru didalam folder Praktikum-5, dan nama foldernya adalah tugas, yang nanti semua tugas kita simpan didalam folder tersebut.

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ mkdir tugas
```

Setelah membuat folder tugas, kita akan masuk ke dalam folder tugas dengan menggunakan command cd.

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ cd tugas
```

Setelah berpindah ke dalam folder tugas, kita akan membuat file untuk membuat tugas 1, dan nama filenya adlaah tugas-1.js.

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5/tugas (main)
$ touch tugas-1.js
```

Setelah membuatnya, kita akan masuk ke file tugas-1.js dan isi didalamnya dengan code-code seperti yang di tugas 1 suruh.

Praktikum-5 > tugas > JS tugas-1.js > intersectionCepat

```javascript
function ukurWaktu(label, fn) {
  const start = Date.now();

  fn();

  const end = Date.now();

  console.log(label, ":", end - start, "ms");
}

function intersectionLambat(arr1, arr2) {
  const hasil = [];

  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j]) {
        hasil.push(arr1[i]);
        break;
      }
    }
  }

  return hasil;
}

function intersectionCepat(arr1, arr2) {
  const set = new Set(arr1);

  return arr2.filter((x) => set.has(x));
```

```
}

console.log("\n=== Fungsi A: Intersection ===");

const arrA1 = Array.from({ length: 5000 }, (_, i) => i);
const arrA2 = Array.from({ length: 5000 }, (_, i) => i + 2500);

ukurWaktu("Intersection Lambat O(n²)", () => intersectionLambat(arrA1, arrA2));

ukurWaktu("Intersection Cepat O(n)", () => intersectionCepat(arrA1, arrA2));

function kelompokAnagram(arr) {
  const map = {};

  for (let kata of arr) {
    const key = kata.split("").sort().join("");

    if (!map[key]) {
      map[key] = [];
    }

    map[key].push(kata);
  }

  return Object.values(map);
}

console.log("\n=== Fungsi B: Anagram ===");
```

```
const words = ["eat", "tea", "tan", "ate", "nat", "bat"];

console.log(kelompokAnagram(words));

const randomWords = Array.from({ length: 5000 }, () =>
  Math.random().toString(36).substring(2, 6),
);

ukurWaktu("Kelompok Anagram", () => kelompokAnagram(randomWords));

function cekKuadratLambat(arr) {
  const n = arr.length;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      for (let k = 0; k < n; k++) {
        if (arr[i] + arr[j] === arr[k] * arr[k]) {
          return true;
        }
      }
    }
  }

  return false;
}

function cekKuadratCepat(arr) {
  arr.sort((a, b) => a - b);
```

```
  const setKuadrat = new Set(arr.map((x) => x * x));

  const n = arr.length;

  for (let i = 0; i < n; i++) {
    for (let j = i + 1; j < n; j++) {
      const jumlah = arr[i] + arr[j];

      if (setKuadrat.has(jumlah)) {
        return true;
      }
    }
  }

  return false;
}

console.log("\n=== Fungsi C: Kuadrat ===");

const arrC = Array.from(
  { length: 200 },
  () => Math.floor(Math.random() * 100) + 1,
);

ukurWaktu("Kuadrat Lambat O(n³)", () => cekKuadratLambat(arrC));

ukurWaktu("Kuadrat Cepat O(n log n)", () => cekKuadratCepat(arrC));
```

4. Tugas 2

Lakukan hal yang sama untuk tugas 2, buat file didalam folder tugas dengan menggunakan command tiuch, lalu mengisi file tersebut dengan code yang sesuai yang di minta.

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5/tugas (main)
$ touch tugas-2.js
```

Isi filenya :

```
Praktikum-5 > tugas > JS tugas-2.js > ...
function fn_O1(n) {
  return n + 1;
}

function fn_On(n) {
  let jumlah = 0;

  for (let i = 0; i < n; i++) {
    jumlah += i;
  }

  return jumlah;
}

function fn_OnLogn(n) {
  let count = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 1; j < n; j *= 2) {
      count++;
    }
  }

  return count;
}

function fn_On2(n) {
  let count = 0;

```

```javascript
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }

  return count;
}

function benchmarkSemua(ukuranData) {
  console.log("=== Benchmark Big O ===");

  ukuranData.forEach((n) => {
    console.log(`\nUkuran data n = ${n}`);

    let start = Date.now();
    fn_O1(n);
    let end = Date.now();

    console.log("O(1):", end - start, "ms");

    start = Date.now();
    fn_On(n);
    end = Date.now();

    console.log("O(n):", end - start, "ms");

    start = Date.now();
    fn_OnLogn(n);
    end = Date.now();

    console.log("O(n log n):", end - start, "ms");

    start = Date.now();
    fn_On2(n);
    end = Date.now();

    console.log("O(n²):", end - start, "ms");
  });
}

benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

2.2.4 OUTPUTNYA

1. Intro-kompleksitas.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksesekusi ===
O(1) jumlahkanRumus: 0 ms
O(n) jumlahkanLinear: 2 ms
O(n^2) cariPasangan: 28 ms

Hasil sama? true
```

2. Kelas-big-o.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ node 02-kelas-big-o.js
=== O(1) - selalu cepat ===
10
true

=== O(log n) - Binary Search ===
 Ditemukan di index 731452 setelah 19 langkah

=== O(n) - Bubble Sort (kecil saja) ===
Max dari 1000 elemen: 999

=== O(n²) — Bubble Sort (kecil saja) ===
[
  64, 34, 25, 12,
  22, 11, 90
]

=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2ⁿ):
135 ms
Waktu fib(35) O(n) memoization:
0 ms
```

3. Space-complexity.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

4. Space-complexity.js + latihan 2

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7

=== Latihan 2 : ===

=== Uji Array Kecil ===
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]

=== Uji Waktu Array Besar ===

Mengukur waktu fungsi Lambat...
Waktu Lambat: 1238 ms

Mengukur waktu fungsi Cepat...
Waktu Cepat: 6 ms
```

5. Refactor.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n²): 0 ms
Duplikat CEPAT O(n) : 1 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

6. Latihan-1.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5 (main)
$ node latihan-1.js
=== Fungsi A ===
Waktu eksekusi untuk n=1000: 0 ms

=== Fungsi B ===
Waktu eksekusi untuk n=1000: 3 ms

=== Fungsi C ===
Waktu eksekusi untuk n=1000: 0 ms

=== Fungsi D ===
Waktu eksekusi untuk n=50: 2 ms
```

7. Tugas-1.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5/tugas (main)
$ node tugas-1.js
=== Fungsi A: Intersection ===
Intersection Lambat O(n²) : 17 ms
Intersection Cepat O(n) : 1 ms

=== Fungsi B: Anagram ===
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
Kelompok Anagram : 15 ms

=== Fungsi C: Kuadrat ===
Kuadrat Lambat O(n³) : 0 ms
Kuadrat Cepat O(n log n) : 0 ms
```

8. Tugas-2.js

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010089_Algoritma_Struktur_Data/Praktikum-5/tugas (main)
$ node tugas-2.js
=== Benchmark Big O ===

Ukuran data n = 100
O(1): 0 ms
O(n): 0 ms
O(n log n): 1 ms
O(n²): 0 ms

Ukuran data n = 500
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 1 ms

Ukuran data n = 1000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 4 ms

Ukuran data n = 5000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 56 ms

Ukuran data n = 10000
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 150 ms
```

BAB III

PENUTUP

3.1 KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa kompleksitas algoritma merupakan ukuran penting untuk menentukan efisiensi suatu program, baik dari segi waktu maupun penggunaan memori.

Penggunaan Big-O Notation memudahkan dalam menganalisis dan membandingkan performa algoritma berdasarkan pertumbuhan input. Selain itu, terdapat berbagai kelas kompleksitas seperti O(1), O(n), hingga O(n!) yang menunjukkan tingkat efisiensi yang berbeda-beda.

Refactoring juga menjadi bagian penting dalam pengembangan program karena dapat meningkatkan kualitas kode tanpa mengubah fungsi utamanya. Dengan memahami konsep-konsep tersebut, programmer dapat membuat program yang lebih optimal, efisien, dan mudah dikembangkan di masa depan.